CSP – Assignment 1

Student Name	Unnatesh Hetampuria
Student ID	2025em1100218

Part A: IAM Setup

An IAM user named Unnatesh was created with console access enabled. Only the minimum required permissions were attached, following the principle of least privilege. All subsequent steps in this assignment were completed exclusively as the Assignment user (not root).

Screenshot 1 — IAM User Details

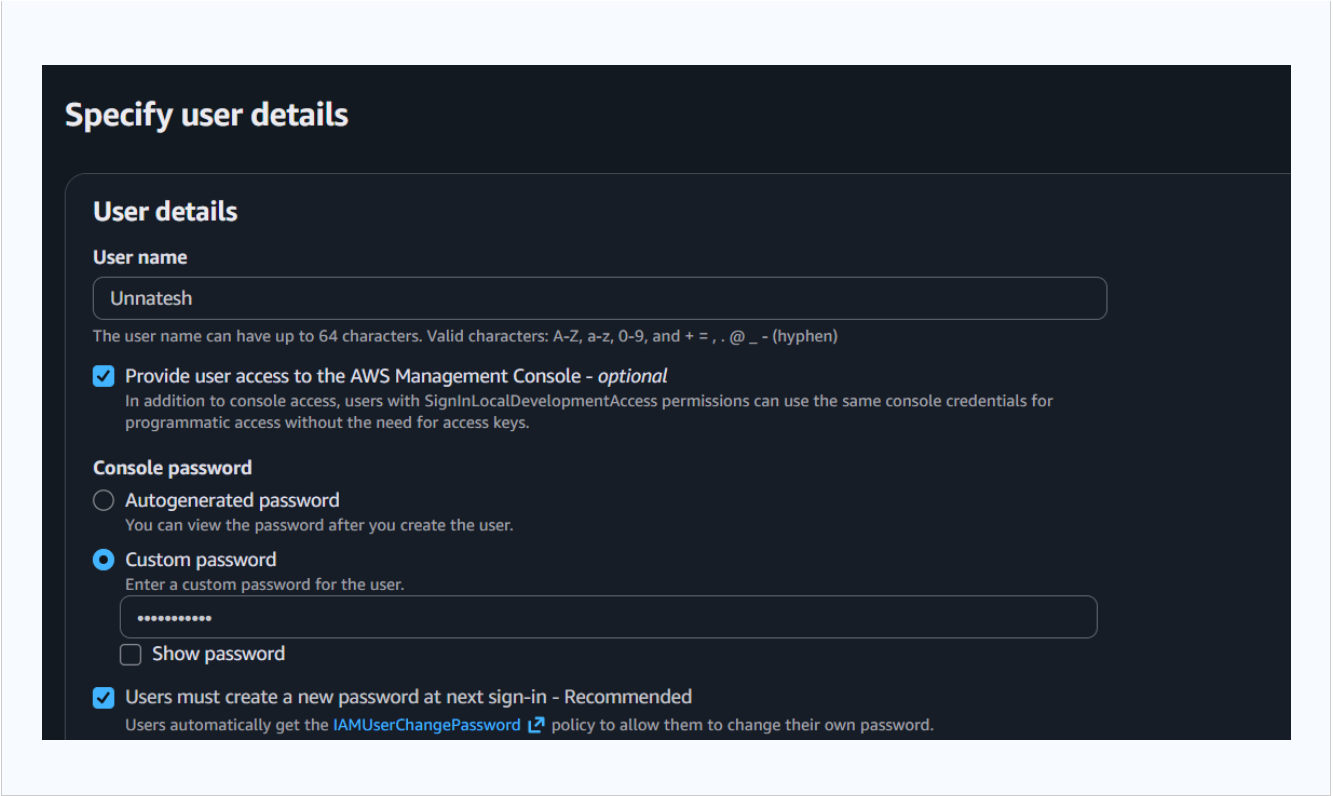


Figure 1: IAM user 'Assignment' created with console access enabled

Screenshot 2 — IAM Permissions

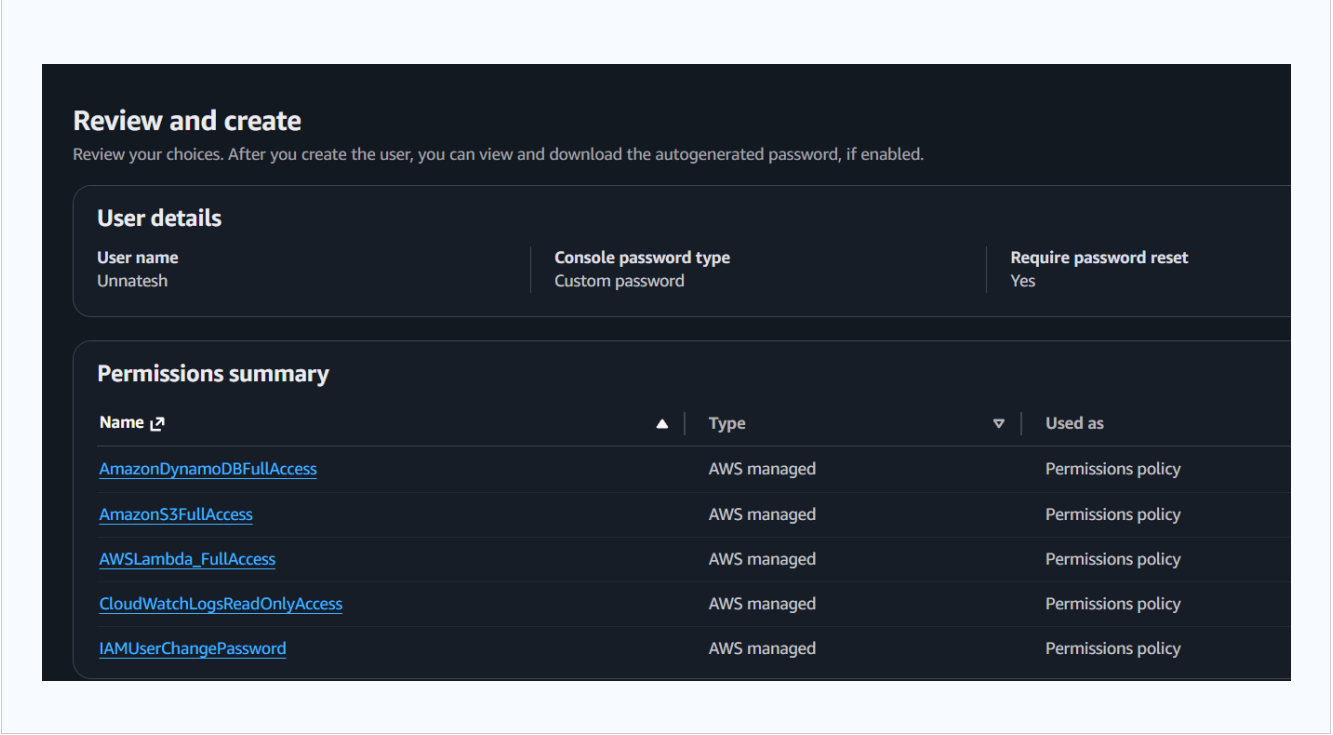


Figure 2: Least-privilege permissions attached to the Assignment IAM user

---

Part B: Architecture Design

Architecture Diagram

The diagram below illustrates the end-to-end event-driven serverless pipeline built on AWS:

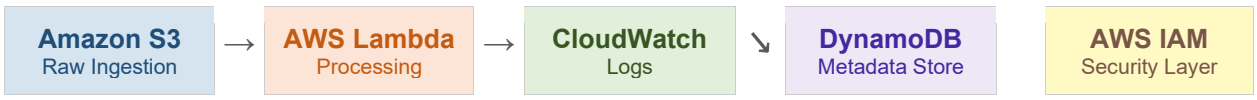


Figure 3: High-level architecture — User Upload → S3 → Lambda → CloudWatch + DynamoDB

Explanation of the Architecture

How the Services Interact

When I upload a CSV file to the designated Amazon S3 bucket, S3 automatically emits a PUT event notification. This event triggers the AWS Lambda function, which reads the file contents directly from S3 using the boto3 SDK. The Lambda function then computes file-level metrics (row count, column count, file size and timestamp) and performs two actions simultaneously: it prints a structured JSON log entry to Amazon CloudWatch Logs for observability, and it writes a metadata record to the Amazon DynamoDB table (FileProcessingMetadata) for persistent storage.

How Security is Enforced

Security is enforced at two levels. First, a dedicated IAM user with only the necessary permissions is used instead of root, reducing the blast radius of any misconfiguration. Second, the Lambda function operates under an IAM execution role that grants read access (s3:GetObject) to S3, write access to DynamoDB, and permission to publish logs to CloudWatch — and nothing more. This ensures that even if the function were compromised, an attacker would have no ability to modify S3 data, access other AWS services, or escalate privileges.

Why Event-Driven Design is Used

An event-driven architecture was chosen because it is inherently cost-efficient and scalable. Lambda runs only when a file is actually uploaded — there is no server polling or idle compute cost. The system scales automatically with upload volume: ten simultaneous uploads trigger ten parallel Lambda invocations without any manual provisioning. This design also decouples the ingestion layer (S3) from the processing layer (Lambda), making it easy to extend the pipeline in future — for example, by adding SNS notifications or Step Functions workflows — without modifying existing components.

Part C: AWS Resource Setup

1. Amazon S3 — Data Ingestion

An S3 bucket was created to serve as the entry point for raw CSV data. Dataset files were uploaded to the bucket to trigger the processing pipeline.

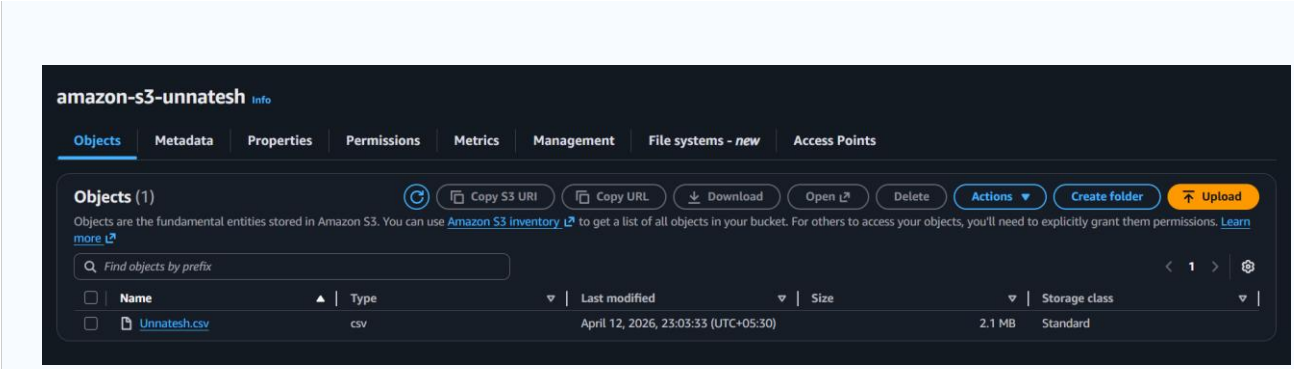


Figure 4: S3 bucket with uploaded CSV dataset file(s)

2. AWS Lambda — Serverless Processing

A Lambda function was created using the Python runtime. The provided baseline script was deployed without modification. An S3 trigger was configured so that any new object uploaded to the bucket automatically invokes the function.

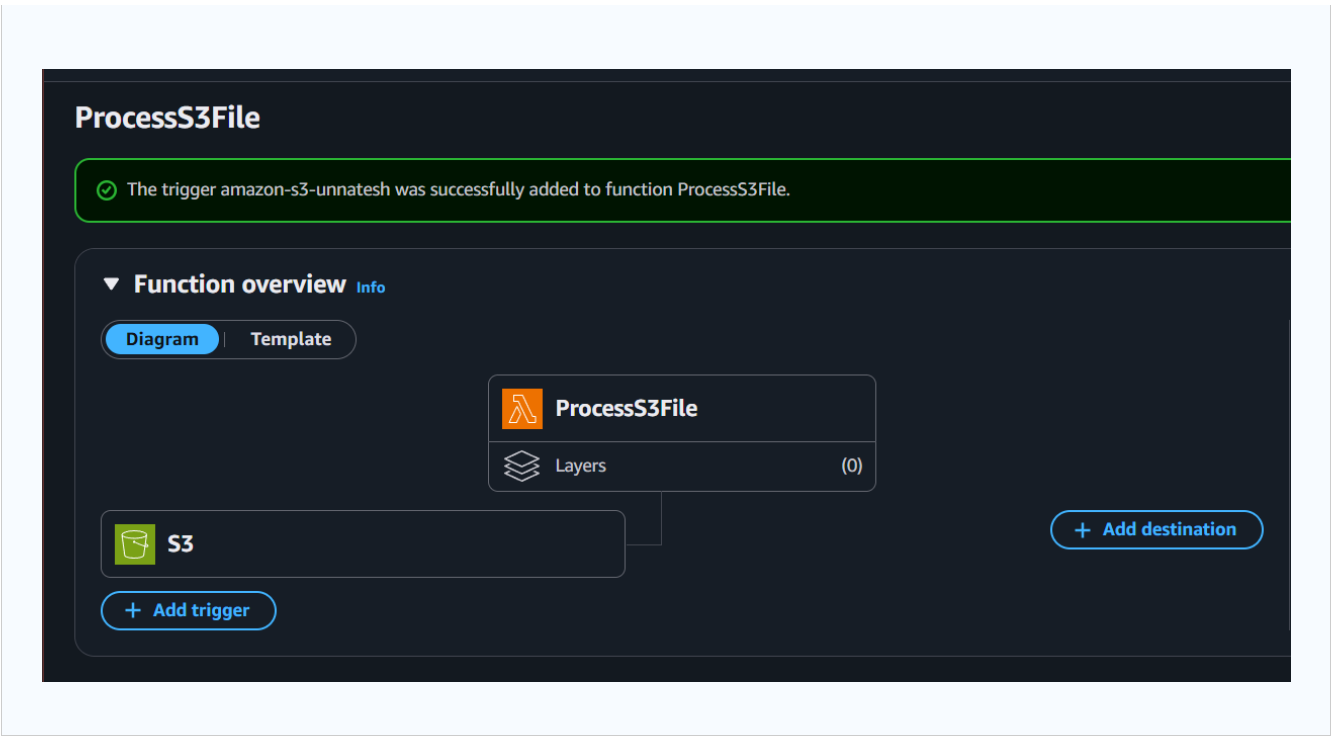
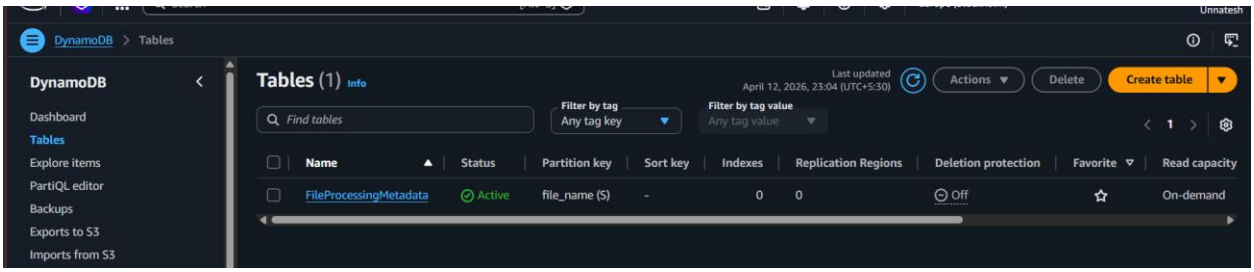


Figure 5: S3 configured as a trigger for the Lambda function

3. Amazon DynamoDB — Metadata Storage

A DynamoDB table named FileProcessingMetadata was created with file_name (String) as the partition key. The table was initially empty; items are populated automatically by Lambda each time a file is processed.



Part D: Output Validation

CloudWatch Logs — Processing Metrics

The Lambda function logged structured JSON metrics to CloudWatch after each file upload, confirming successful processing.

Figure 6: CloudWatch log entry showing file metrics (name, size, rows, columns, timestamp)

DynamoDB Table — Populated Entries

After uploading the dataset file(s), the DynamoDB table was automatically populated with one entry per file, confirming end-to-end pipeline functionality.

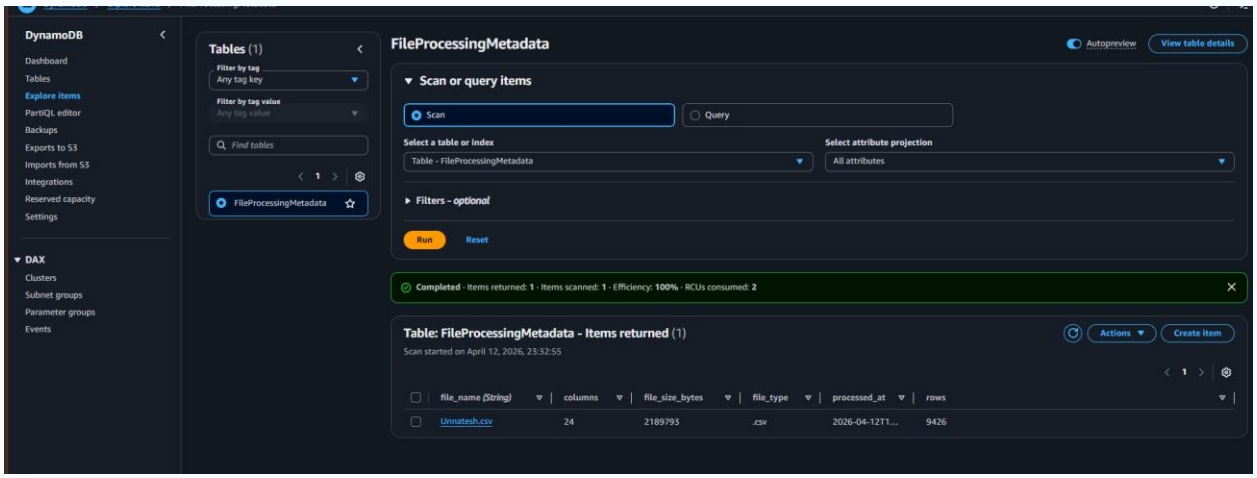


Figure 7: DynamoDB table entries created by Lambda after file upload

Reflection and Insights

Building this serverless pipeline provided hands-on experience with the practical challenges of cloud integration. The most significant learning was around IAM role configuration — it was not immediately obvious which permissions Lambda requires to read from S3 and write to DynamoDB, and iterating on the execution role reinforced the importance of testing with least-privilege before broadening access.

An encoding issue with non-UTF-8 CSV files was encountered and resolved by handling decoding errors in the Lambda function.

The event-driven trigger model was straightforward to configure but highlighted an important design consideration: Lambda functions must be idempotent if S3 can re-deliver events (e.g., on network retries). For a production system, adding a condition check on DynamoDB before writing would prevent duplicate entries.

CloudWatch proved invaluable for debugging — structured JSON logs made it easy to verify that the correct bucket name, file key, and row counts were being captured. Going forward, adding CloudWatch Alarms to detect Lambda errors or unusually large files would improve the observability of the pipeline significantly.

This assignment demonstrated me how loosely coupled AWS services can be orchestrated into a scalable, secure, and automated data processing pipeline with minimal infrastructure overhead